Hard DP

SOS DP, broken profile, the Aliens trick, slope trick

Learning goals

- Compute sums over subsets/supersets in $O(n\,2^n)$; OR/AND and **subset convolution**
- Write **broken-profile** DP cell by cell; know when to switch to a transfer matrix
- Remove a "exactly k" dimension with the **Aliens trick** (WQS / Lagrangian relaxation) — including the tie-breaking that makes it correct
- Represent convex piecewise-linear DP states with **slope trick** heaps

Where we are

- Week 26: bitmask DP · Week 27: interval & digit DP
- Regional-05: prefix-sum / monotonic-queue optimisations
- Nationals-03: CHT, Li Chao, divide & conquer DP, Knuth
- WF-03: LP duality — **Aliens is Lagrangian duality** for a concave 1-parameter family
- Today: four techniques that each turn an infeasible DP into an easy one

SOS DP: the problem

Given f over masks of n bits, compute for every S:

$F(S) = \sum_{T \subseteq S} f(T)$

- Naive over all pairs: 4^n; over submasks: $\sum 2^{|S|} = \mathbf{3^n}$
- n = 20: $3^{20} \approx 3.5 \cdot 10^9$ × — $n \cdot 2^n \approx 2 \cdot 10^7$ ✓
- Idea: F is an **n-dimensional prefix sum** over the hypercube $\{0,1\}^n$
- Do prefix sums one dimension at a time, like 2D prefix sums

SOS DP: derivation

After processing bits 0..i−1:

$F_i(S) = \Sigma f(T)$ over $T \subseteq S$ that agree with S on bits ≥ i

Processing bit i: if S has bit i, $F_{i+1}(S) = F_i(S) + F_i(S \setminus \{i\})$.

```
void subsetZeta(vector<ll>& f, int n) {          // f[S] <- sum_{T ⊆ S} f[T]
    for (int i = 0; i < n; i++)
        for (int S = 0; S < (1 << n); S++)
            if (S >> i & 1) f[S] += f[S ^ (1 << i)];
}
void subsetMobius(vector<ll>& f, int n) {        // inverse transform
    for (int i = 0; i < n; i++)
        for (int S = 0; S < (1 << n); S++)
            if (S >> i & 1) f[S] -= f[S ^ (1 << i)];
}
```

Superset version: `if (!(S >> i & 1)) f[S] += f[S | 1 << i]`.

Transforms you get for free

Want h[S] = Σ ...	Transform
f[A]·g[B] over A ∪ B = S (OR conv.)	subset zeta, multiply, Möbius
f[A]·g[B] over A ∩ B = S (AND conv.)	superset zeta, multiply, inverse
f[A]·g[B] over A ⊕ B = S (XOR conv.)	Walsh–Hadamard (WF-02)
f[A]·g[S \ A] over A ⊆ S (**subset conv.**)	ranked zeta: $O(n^2\,2^n)$

Keep the **bit loop outermost**; the order of masks inside does not matter (step i only combines masks that differ in bit i).

Subset convolution

Disjointness: $A \cup B = S$ **and** $|A| + |B| = |S|$.

```
vector<ll> subsetConv(const vector<ll>& f, const vector<ll>& g, int n) {
    int N = 1 << n;
    vector<vector<ll>> F(n + 1, vector<ll>(N)), G = F, H = F;
    for (int S = 0; S < N; S++)
        F[__builtin_popcount(S)][S] = f[S], G[__builtin_popcount(S)][S] = g[S];
    for (int k = 0; k <= n; k++) subsetZeta(F[k], n), subsetZeta(G[k], n);
    for (int k = 0; k <= n; k++)
        for (int j = 0; j <= k; j++)
            for (int S = 0; S < N; S++) H[k][S] += F[j][S] * G[k - j][S];
    for (int k = 0; k <= n; k++) subsetMobius(H[k], n);
    vector<ll> h(N);
    for (int S = 0; S < N; S++) h[S] = H[__builtin_popcount(S)][S];
    return h;
}
```

Use it for "partition the set into blocks" DPs: exp of a set power series.

SOS in the wild

- CF 165E *Compatible Numbers*: for each a, find b with a & b = 0 ⇒ "any element ⊆ ~a": SOS with "witness" instead of sum
- CF 449D *Jzzhu and Numbers*: #subsets with AND = 0 ⇒ superset counts c(S), then inclusion–exclusion Σ (−1)^{|S|} (2^{c(S)} − 1)
- CF 383E *Vowels*: sum over 2^{24} masks of squares of counts
- CF 1208F *Bits And Pieces*: keep the **two largest indices** per superset instead of a sum
- Any **commutative monoid** works in the zeta step (sum, max, top-2, OR)

Broken profile DP

Tile an n × m board; process cells in row-major order.

```
row i   : . . . X X X X    X = cells of row i already decided
row i+1 : Y Y Y . . . .    Y = cells of row i+1 already covered
                ^ current cell (i, j)
```

- **Profile** = m bits: bit k < j refers to (i+1, k), bit k ≥ j refers to (i, k)
- At each cell: decide what covers it, update **one** bit (and maybe bit j+1)
- O(n · m · 2^m) time, O(2^m) memory; put the **smaller** dimension in m

Domino tilings code

```
ll dominoTilings(int n, int m) {
    if (m > n) swap(n, m);
    vector<ll> dp(1 << m, 0), nd(1 << m);
    dp[0] = 1;
    for (int i = 0; i < n; i++) for (int j = 0; j < m; j++) {
        fill(nd.begin(), nd.end(), 0);
        for (int mask = 0; mask < (1 << m); mask++) if (dp[mask]) {
            ll v = dp[mask];
            if (mask >> j & 1) { nd[mask ^ (1 << j)] = (nd[mask ^ (1 << j)] + v) % MOD; continue; }
            if (i + 1 < n)                                          // vertical
                nd[mask | (1 << j)] = (nd[mask | (1 << j)] + v) % MOD;
            if (j + 1 < m && !(mask >> (j + 1) & 1))                // horizontal
                nd[mask | (1 << (j + 1))] = (nd[mask | (1 << (j + 1))] + v) % MOD;
        }
        swap(dp, nd);
    }
    return dp[0];
}
```

8 × 8 → 12 988 816 ✓

Beyond dominoes

- Obstacles: a blocked cell behaves like "already covered"
- Min/max cost tilings: replace `+` by `min`
- Trominoes / L-shapes: may touch 2 rows ahead ⇒ profile per cell of 2 rows, or 3^n states
- **Plug DP** (connectivity profiles): Hamiltonian cycles / paths on grids — profile stores bracket matching of path ends, states hashed
- Huge n (10^{18}) with small m: row-to-row **transfer matrix** T (2^m × 2^m) and fast exponentiation, or Berlekamp–Massey on the sequence (WF-02)

The Aliens trick: setting

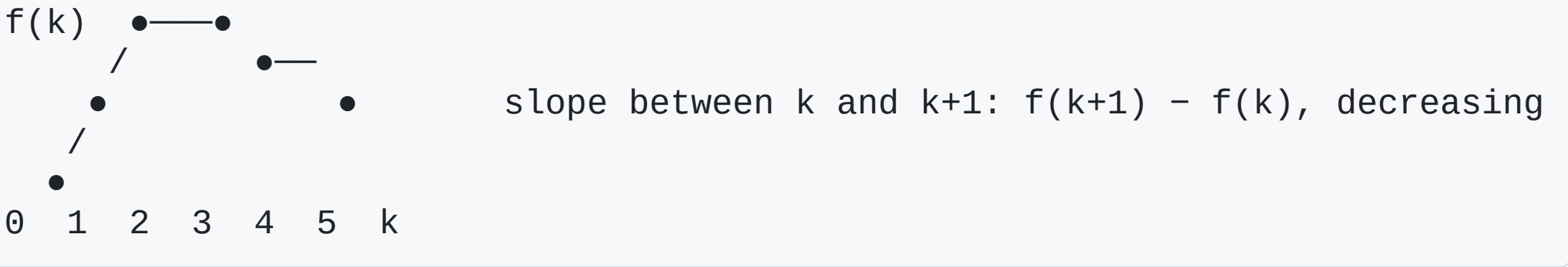
Want f(k) = best value using **exactly k** "items" (segments, groups, cameras…).

- DP with a k-dimension: O(n · k) or worse
- Suppose f is **concave** in k (for maximisation)
- Penalise every item by λ and drop the k-dimension:

  g(λ) = max over all solutions of (value − λ · #items)
- One λ ⇒ one O(n) DP; binary search on λ ⇒ O(n log C)

IOI 2016 *Aliens* gave the trick its name.

Geometry of the trick



- g(λ) = max_k (f(k) − λk): the **supporting line** of slope λ touches the graph at the optimal k
- Larger λ ⇒ fewer items
- For integer data the slopes are integers ⇒ integer λ suffices
- Recover f(k) = g(λ) + λ·k for a λ at which k is **among** the optimal counts

Ties: the part everyone gets wrong

If f has a straight piece, several k are optimal for the same λ.

- Make the DP return, among optimal solutions, the **minimum** count (consistent tie-break!)
- Binary search the **smallest λ** with cnt_min(λ) ≤ k
- At that λ, k is optimal (k lies on the tangent segment)
- Answer = g(λ) + λ · **k** — not λ · cnt

Mixing "min count" in one place and "max count" in another gives off-by-one-segment answers.

Aliens code

```
pair<ll,int> solve(const vector<ll>& a, ll lambda) {   // max sum of subarrays, each costs lambda
    auto better = [](pair<ll,int> x, pair<ll,int> y) {
        return x.first != y.first ? x.first > y.first : x.second < y.second; };
    pair<ll,int> out = {0, 0}, in = {LLONG_MIN / 4, 0};
    for (ll x : a) {
        pair<ll,int> from = better(out, in) ? out : in;
        pair<ll,int> start = {from.first - lambda + x, from.second + 1}, ext = {in.first + x, in.second};
        in = better(start, ext) ? start : ext;
        out = from;
    }
    return better(out, in) ? out : in;
}
ll exactlyK(const vector<ll>& a, int k) {
    ll B = 1; for (ll x : a) B += llabs(x);              // all slopes lie in [-B, B]
    ll lo = -B, hi = B;
    while (lo < hi) { ll mid = lo + (hi - lo) / 2;
        if (solve(a, mid).second <= k) hi = mid; else lo = mid + 1; }
    return solve(a, lo).first + lo * k;
}
```

Overflow: $\lambda \cdot$ count can reach $n \cdot \Sigma|a_i|$ — use `__int128` for the value when that exceeds $9 \cdot 10^{18}$.

Proving concavity

You must convince yourself f is concave — otherwise the trick silently fails.

- **Flow / matroid**: f(k) is the value of k augmentations of a min-cost flow ⇒ concave (exactly-k disjoint subarrays)
- **Quadrangle inequality** on the cost of a partition DP ⇒ convex in #parts (IOI 2016 Aliens, partition into k groups)
- **Exchange argument**: from optimal solutions for k−1 and k+1 build two for k
- When unsure: brute force f(k) for small random inputs and **check the second differences**

Aliens + other optimisations

- Inner DP still too slow? Combine: Aliens removes k, then **CHT / Li Chao** (nationals-03) speeds up the inner $O(n^2) \Rightarrow O(n \log n \log C)$
- IOI 2016 *Aliens*: Aliens + CHT
- CF 739E *Gosha is hunting*: Aliens on one resource (or on both, nested) + a small DP
- CF 1279F *New Year and Handle Change*: Aliens over "number of operations"
- λ can be fractional for real-valued problems: binary search on doubles, take care with ties

Slope trick: idea

Many DPs have the form dp_i(x) = (cost_i(x)) + min over a window of dp_{i−1}.

- If every dp_i is **convex piecewise linear with integer breakpoints**, store it as
  - min value `minf`
  - multiset **L** of breakpoints left of the minimum (slope changes by −1 each)
  - multiset **R** of breakpoints right of the minimum
- f(x) = minf + Σ_{l∈L} (l − x)⁺ + Σ_{r∈R} (x − r)⁺
- Heaps + lazy shifts ⇒ each operation O(log n)

Slope trick operations

Operation	Implementation
add $(x - a)^+$	minf += $(\text{top } L - a)^+$; push a to L; move top L to R
add $(a - x)^+$	minf += $(a - \text{top } R)^+$; push a to R; move top R to L
add $\lvert x - a \rvert$	both of the above
$f(x) \leftarrow \min_{y \le x} f(y)$ (prefix min)	clear R
$f(x) \leftarrow \min_{y \ge x} f(y)$ (suffix min)	clear L
$f(x) \leftarrow \min_{x-b \le y \le x-a} f(y)$	lazy: L += a, R += b

Slope trick code

```
struct SlopeTrick {
    ll minf = 0, addL = 0, addR = 0;
    priority_queue<ll> L;                                  // max-heap
    priority_queue<ll, vector<ll>, greater<ll>> R;         // min-heap
    ll topL() const { return L.empty() ? LLONG_MIN / 4 : L.top() + addL; }
    ll topR() const { return R.empty() ? LLONG_MAX / 4 : R.top() + addR; }
    void pushL(ll a) { L.push(a - addL); }
    void pushR(ll a) { R.push(a - addR); }
    ll popL() { ll v = topL(); L.pop(); return v; }
    ll popR() { ll v = topR(); R.pop(); return v; }
    void addXminusA(ll a) { minf += max(0LL, topL() - a); pushL(a); pushR(popL()); }
    void addAminusX(ll a) { minf += max(0LL, a - topR()); pushR(a); pushL(popR()); }
    void addAbs(ll a) { addXminusA(a); addAminusX(a); }
    void prefixMin() { while (!R.empty()) R.pop(); }
    void suffixMin() { while (!L.empty()) L.pop(); }
    void slideMin(ll a, ll b) { addL += a; addR += b; }    // a <= b
};
```

Example: CF 713C

> Make the array **strictly increasing** with minimum $\Sigma |a_i - b_i|$.

- Substitute $c_i = a_i - i \Rightarrow$ non-decreasing
- dp_i(x) = |x – c_i| + min_{y ≤ x} dp_{i–1}(y)

```
SlopeTrick st;
for (int i = 0; i < n; i++) { st.prefixMin(); st.addAbs(a[i] - i); }
cout << st.minf << '\n';
```

O(n log n), values up to 10^9 — no coordinate compression needed. (The classic "one heap" solution is this code with R discarded.)

More slope trick

- **Window moves** ("each step you can move at most d"): slideMin(−d, d)
- ABC 217H *Snuketoon*: walk on a line, penalties $(x - a)^+$ / $(a - x)^+$ at given times
- APIO 2016 *Fireworks*: functions on a tree — **merge** children's heaps (small-to-large or mergeable heaps), then reshape at each edge
- CF 1534G *A New Beginning*: rotate coordinates, then slope trick
- Need f(x) at the end? Pop all breakpoints and walk the slopes

Pitfalls

- SOS: iterating masks **inside** before bits (`for S` outside `for i`) is wrong for zeta
- Subset convolution: forgetting the rank ⇒ computes OR-convolution
- Broken profile: transitions that write two bits must check the second bit is free
- Aliens: inconsistent tie-breaking; λ range too small (slopes can be up to $\Sigma|a_i|$); using cnt instead of k in the final formula
- Aliens on a non-concave f: passes samples, fails system tests — brute-check concavity
- Slope trick: forgetting lazy offsets when pushing; `LLONG_MIN` sentinels overflowing when shifted

Practice problems

- CF **165E**, CF **449D**, CF **383E**, CF **1208F** — SOS DP
- Library Checker *Subset Convolution*
- CSES **2181** *Counting Tilings*; POJ **2411** *Mondriaan's Dream* — broken profile
- IOI 2016 *Aliens* (oj.uz `IOI16_aliens`); CF **739E**; CF **1279F** — Aliens trick
- CF **713C**, CF **13C** — slope trick basics
- AtCoder ABC 217 H *Snuketoon*; CF **1534G**; APIO 2016 *Fireworks* — slope trick advanced

Summary

- SOS = n-dimensional prefix sums; ranked version = subset convolution
- Broken profile = cell-by-cell frontier DP, O(nm2^m)
- Aliens = Lagrangian relaxation of "exactly k" on a concave f; tie-break consistently, answer g(λ) + λk
- Slope trick = convex piecewise-linear functions as two heaps of breakpoints

Next: Lecture 10 — building the team reference document and a full World Finals simulation